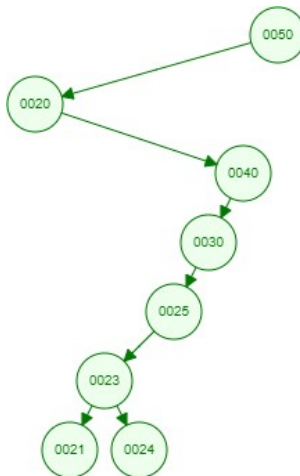


AVL Trees

- An **AVL tree** is a balanced binary search tree(BBST).
- Named after their inventors, Adelson - Velskii and Landis, they were the first dynamically balanced trees to be proposed.

BST:

- Smaller element goes left and larger elements goes right to Parent.
- BST is not a balanced Tree.
- Searching and any operation taking much time in BST.
- Height of the Tree increases when the Tree is not balance.
- In BST, we search for a node level by level

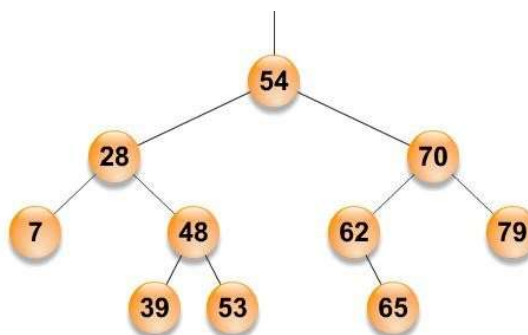


AVL Tree:

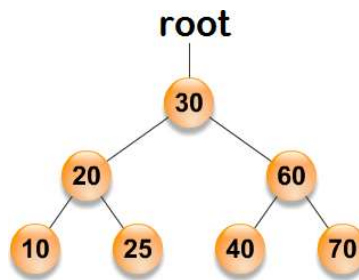
- Solution to Balanced Binary search Tree.
 - An AVL tree follows Binary search tree rules only
 - The sub-trees of every node differ in height by at most one.
 - Every sub-tree is an AVL tree.
- $$\text{abs}(\text{height of right sub tree} - \text{height of left sub tree}) \leq 1$$

Rules of AVL tree:

- Smaller element goes left and larger elements goes right to Parent.

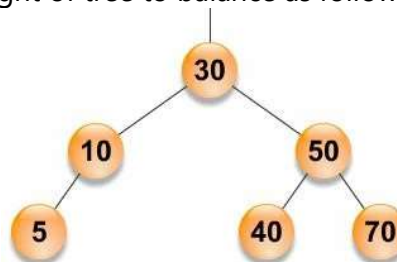


Balanced Tree: Height of Right sub tree – Height of left sub tree = 0

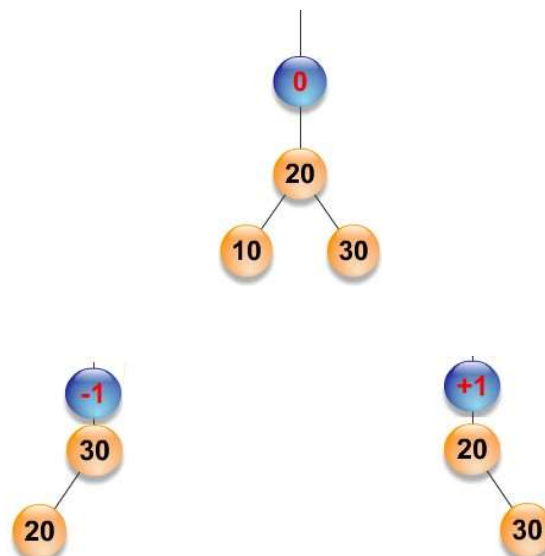


Balanced Tree

- We are not checking the number of nodes in the sub tree.
- We are checking the height of tree to balance as follows:



- If the height of left sub tree or right sub tree is 0 or +1 or -1, the Tree is balanced, we cannot perform any rotations.



[Balnced tree]

[Balnced tree]

Height of right sub tree - Height of left sub tree = +1 or -1

Imbalanced tree: When the sub tree height is either +2 or -2 is called imbalanced tree. We need to perform rotations to balance the Tree.



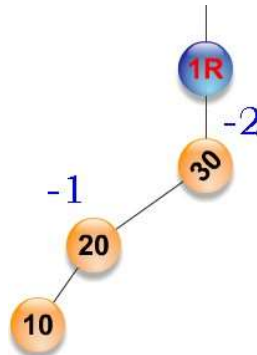
Height of Right sub tree - Height of Left sub tree = +2 or -2

AVL Rotations:

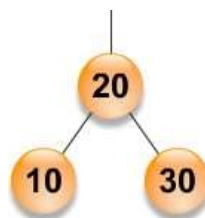
- There are 4 rotations to balance the binary search tree
 - L-L-Rotation
 - R-R-Rotation
 - L-R-Rotation
 - R-L-Rotation

1. L-L rotation:

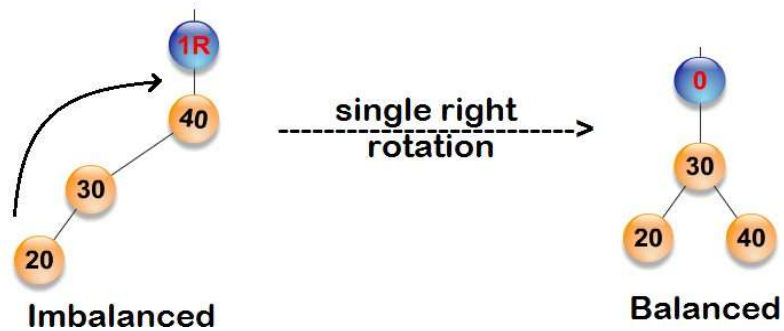
- a. Left Child Left Heavy
- b. Balance Factor of Parent is -2 and Left Child is -1



- c. Single right rotation is required.
- d. After rotation:
 - i. Left Child become Parent
 - ii. Parent become right child

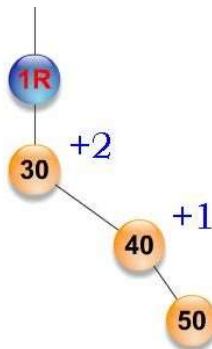


L-L-Rotation : Left child is left heavy



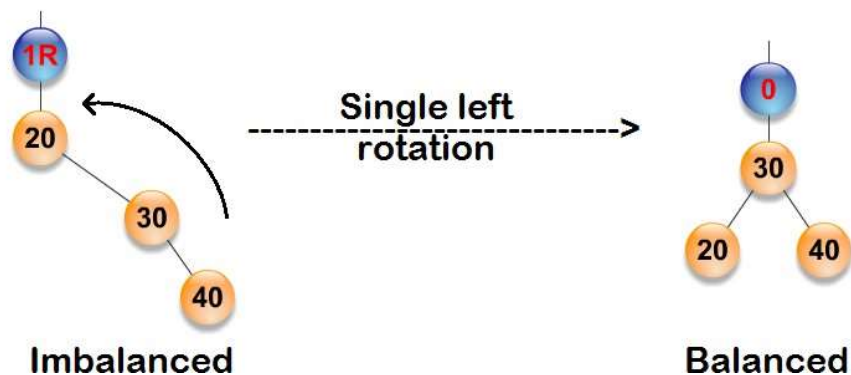
2. R-R rotation:

- Right Child Right Heavy
- Balance Factor of Parent is +2 and Child is +1



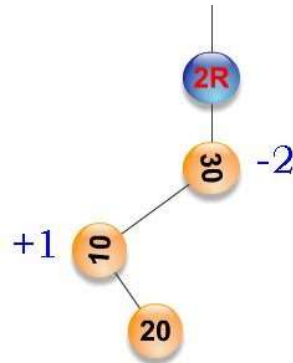
- Single left rotation is required.
- Right Node become Parent and Parent become Left child after rotation

R-R-Rotation : Right child is right heavy



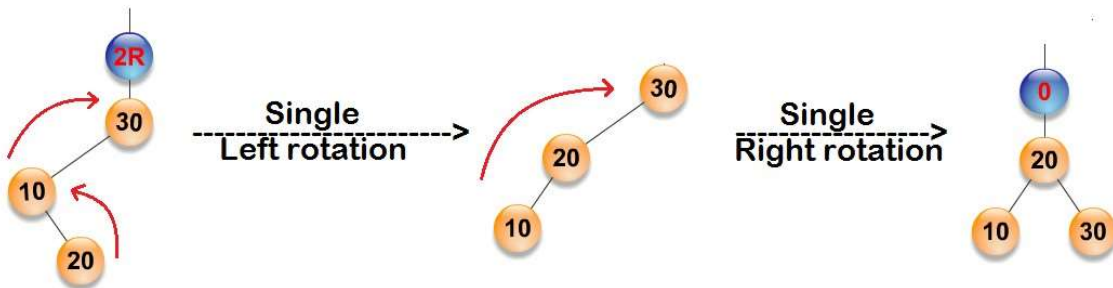
3. L-R rotation:

- a. Left Child is Right Heavy
- b. Balance Factor of Parent is -2
- c. Balance factor of Child is +1



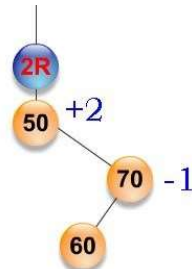
- d. Double rotation is required.
- e. First Single Left rotation
- f. Second Single right rotation

L - R - Rotation : Left child is Right heavy



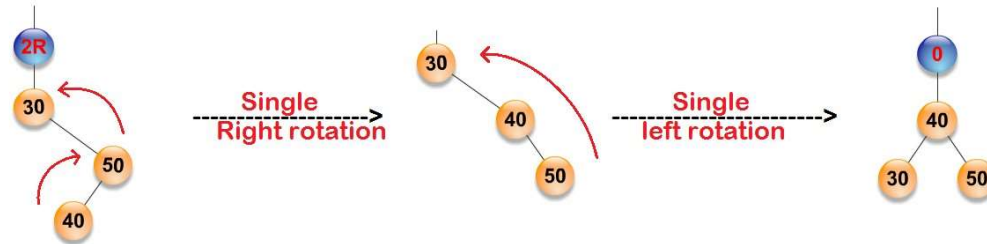
4. R-L rotation:

- a. Right Child Left Heavy
- b. Balance Factor of Parent is +2 and Child is -1

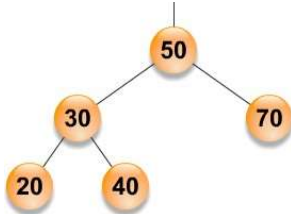


- c. Double rotation is required.
- d. First Single right rotation
- e. Second Single left rotation

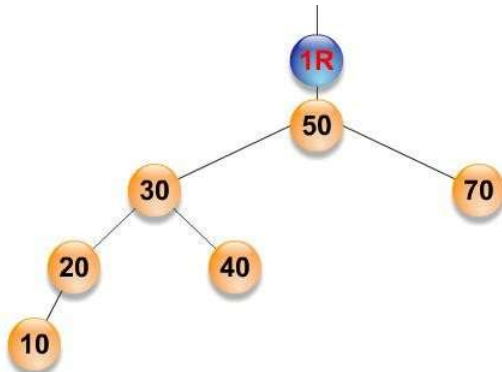
R - L - Rotation : Right child is Left Heavy



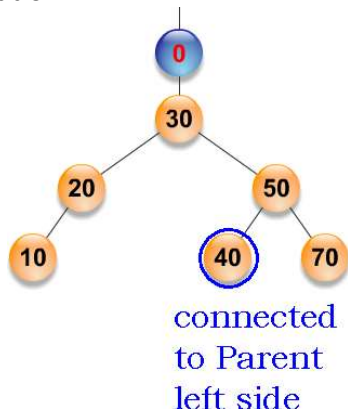
L-L-Rotation:

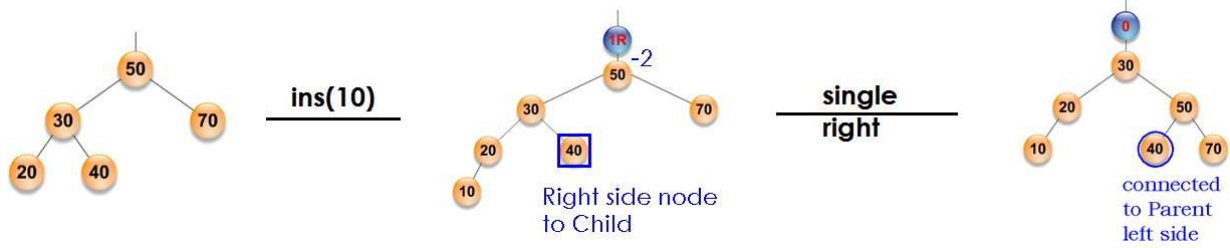


Insert-10:

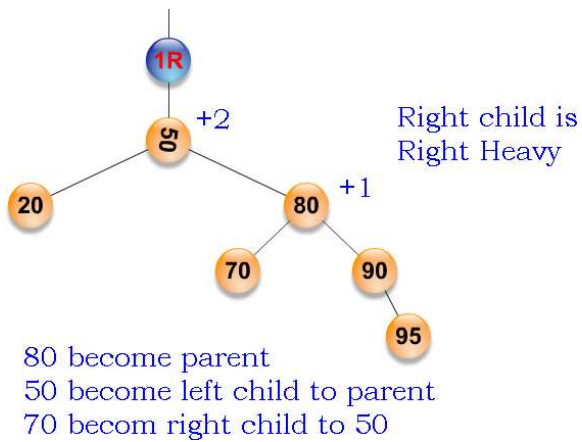
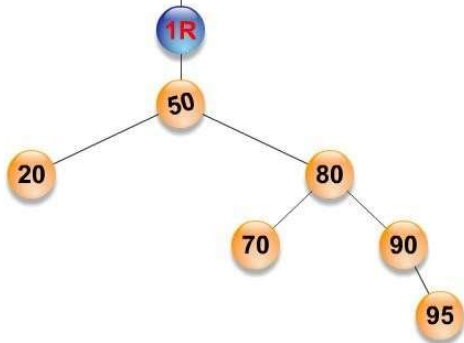


In single right rotation: If the Child has right sub tree, becomes left sub tree to parent after rotation

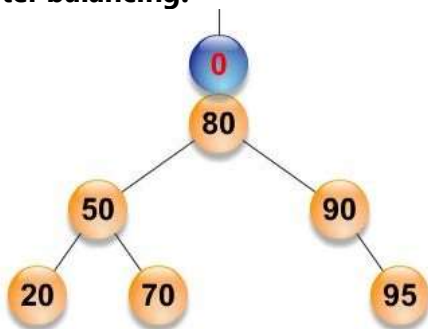


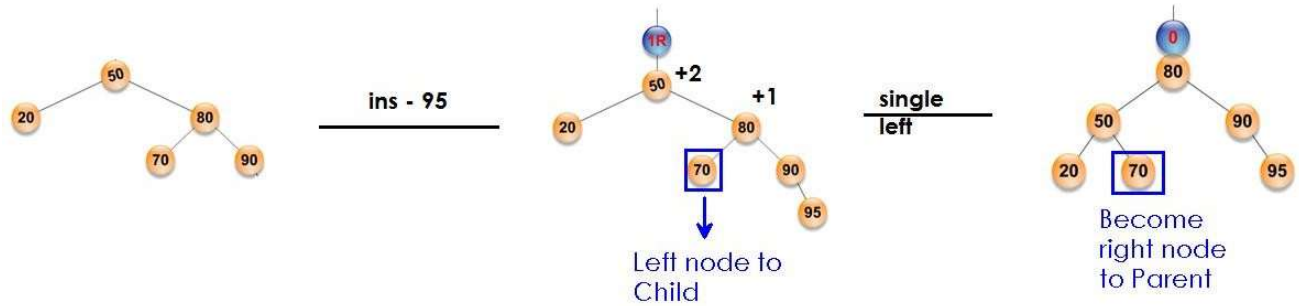


In R-R-Rotation:

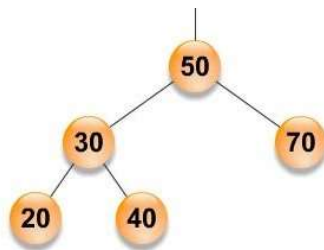


After balancing:

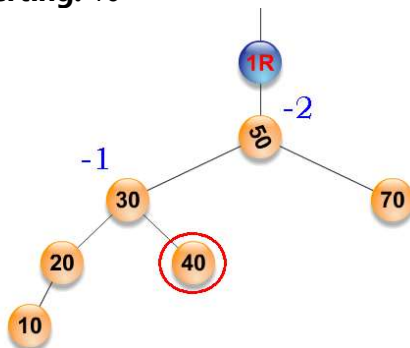




Inserting: 50, 70, 30, 40, 20

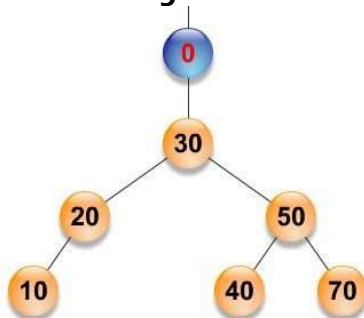


Inserting: 10

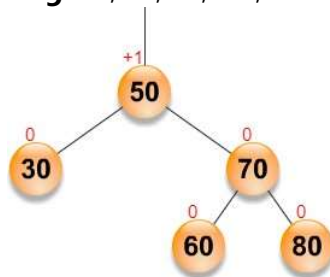


Single right rotation is required
 30 become parent, 50 become right child
 40 become 50 to left.

After Balancing:

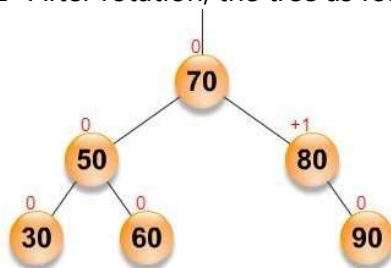


Inserting: 50, 30, 70, 80, 60

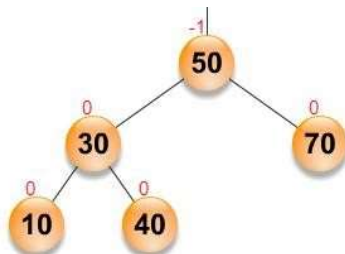


Inserting: 90

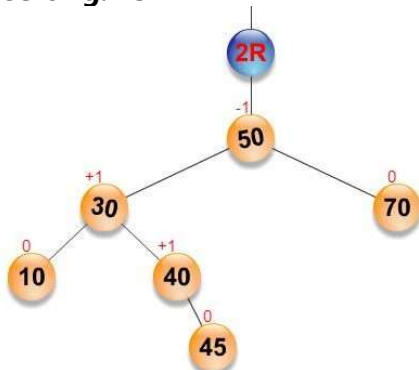
- Right child is Right heavy
- R-R rotation is required.
- After rotation, the tree as follows.



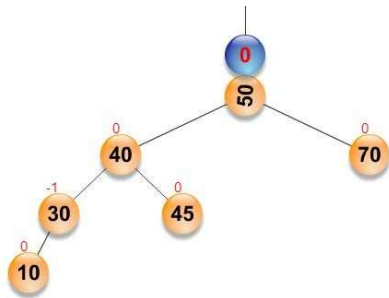
Inserting: 50, 70, 30, 40, 10



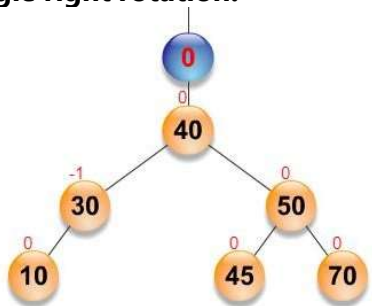
Inserting: 45



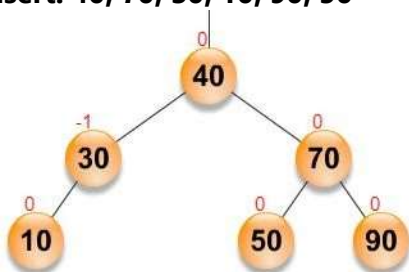
Single Left rotation:



Single right rotation:

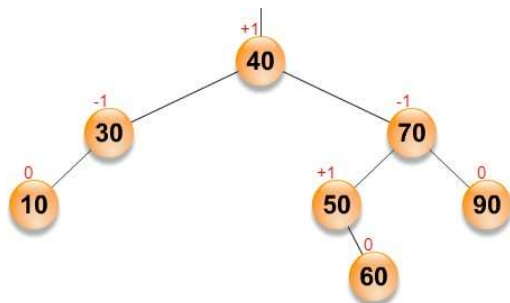


Insert: 40, 70, 30, 10, 90, 50

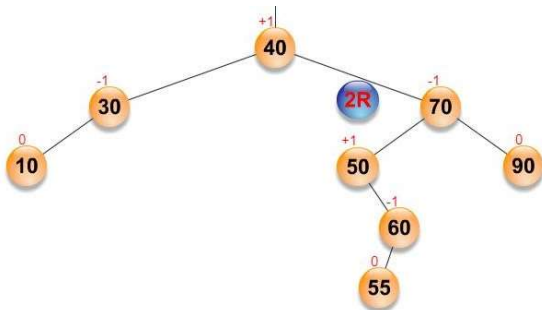


Inserting: 60

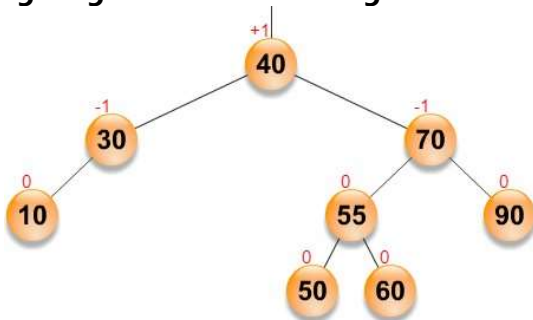
- Balance
- No rotation required



Inserting: 55

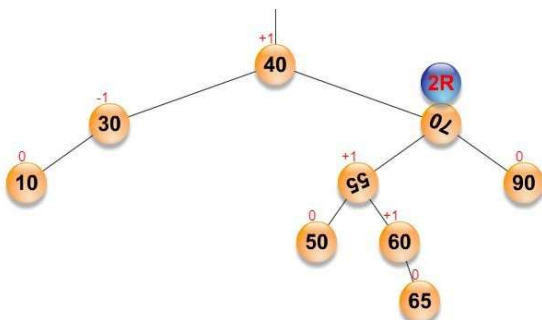


Single right rotation and Single left rotation:

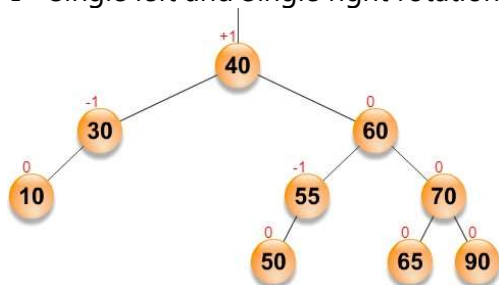


Insert 65:

- ➔ Left child is Right heavy
- ➔ L-R rotation
- ➔ Single left and Single right rotation

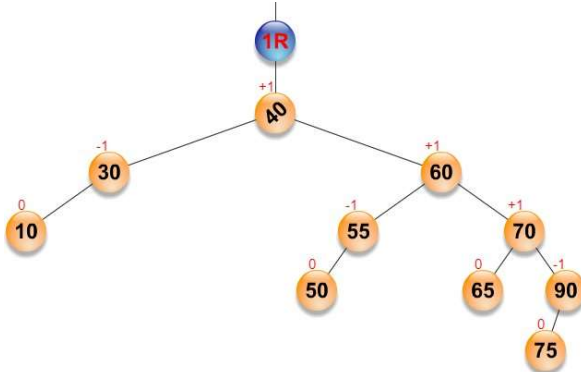


- ➔ Single left and Single right rotation

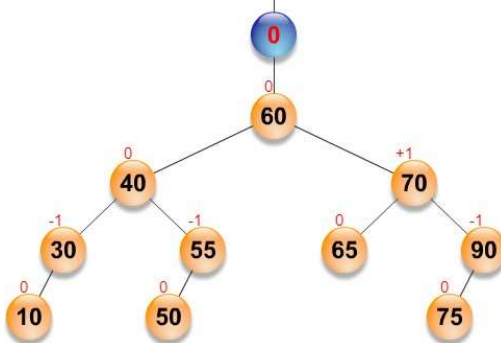


Insert – 75:

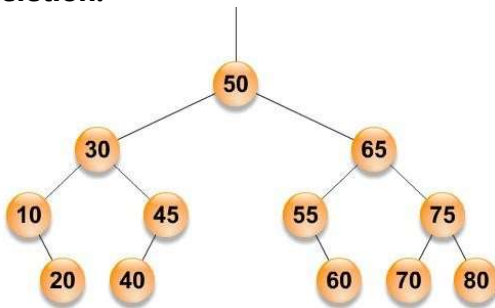
- Right Child is Right heavy
- R-R rotation
- Single left rotation



After single left rotation:

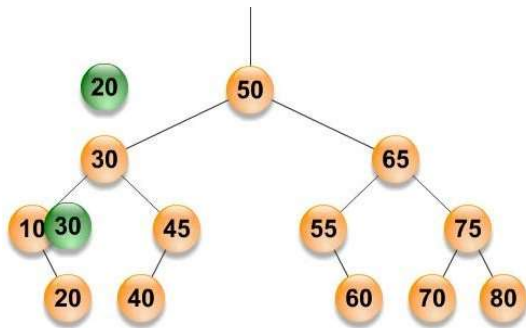


Deletion:

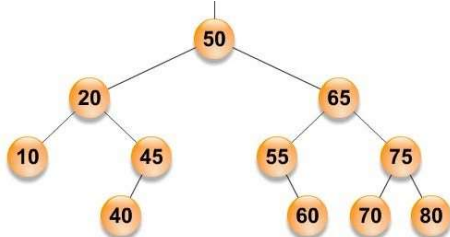


Delete – 30:

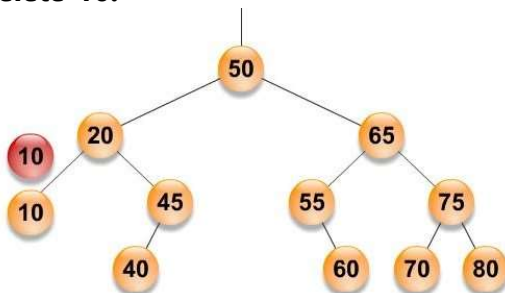
- Deleting a node with 2 children.
- Replace the node(highest value in the left sub tree)



Result: and the Tree is balanced

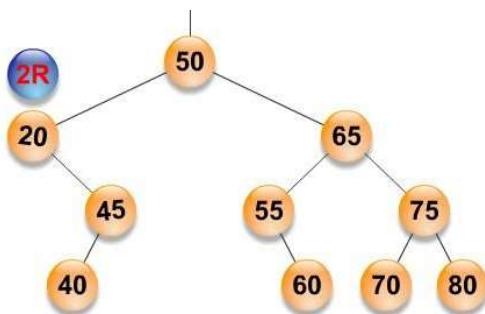


Delete-10:

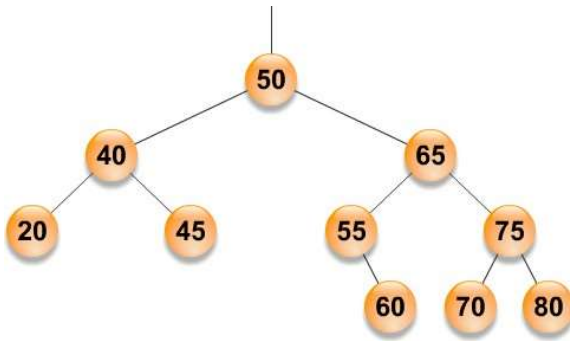


→ Tree is not balanced

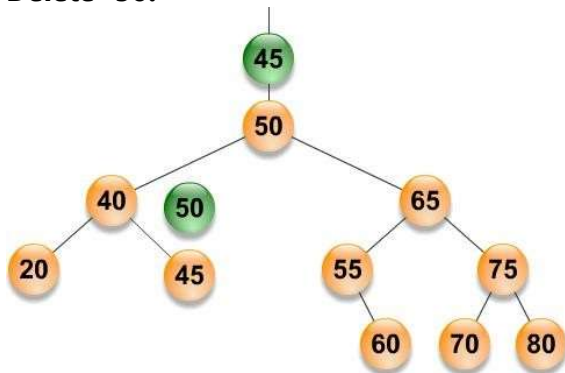
→ Right child of 20 is left heavy -> RL rotation required



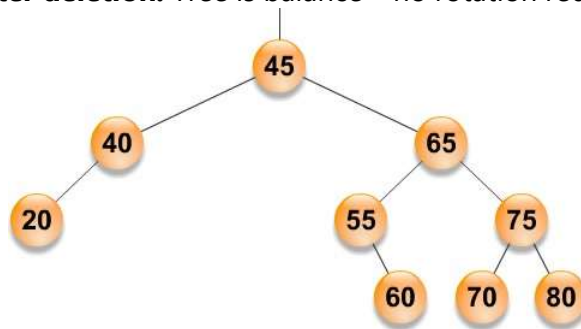
After rotation:



Delete- 50:



After deletion: Tree is balance – no rotation required



Delete-40:

